MODUL 3

Singly Linked List

Modul Praktikum Struktur Data • C++17

Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

1. menjelaskan fungsi node, head, dan next;
2. membentuk singly linked list menggunakan alokasi dinamis;
3. menelusuri seluruh node mulai dari head;
4. menambahkan node pada awal dan akhir list;
5. mencari dan menghapus node berdasarkan data;
6. menangani list kosong, satu node, dan beberapa node; serta
7. membebaskan seluruh node secara benar.

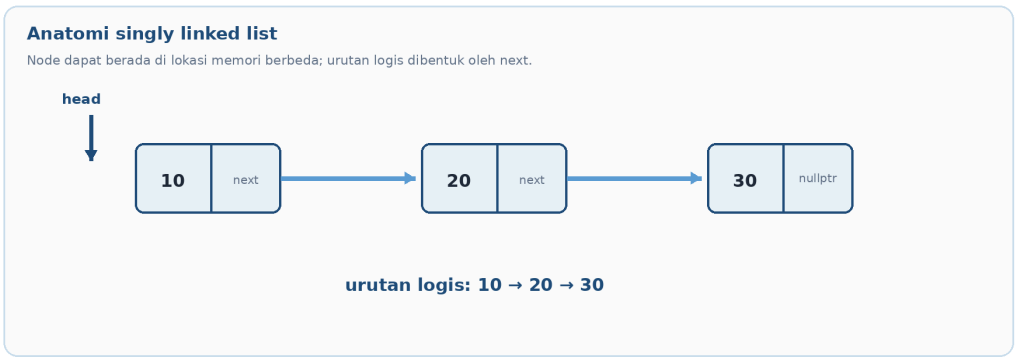
2. Dasar Singly Linked List

Singly linked list menyimpan data pada sejumlah node. Urutan logis dibentuk oleh pointer next yang menghubungkan satu node dengan node berikutnya.

2.1 Struktur node

Setiap node terdiri atas field data dan pointer next. Pointer next berisi alamat node berikutnya atau nullptr jika node tersebut merupakan node terakhir.

```
struct Node {
    int data;
    Node* next;
};
```

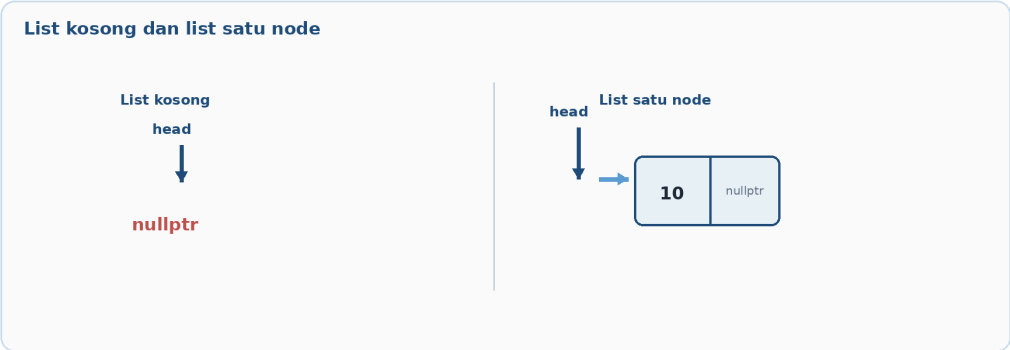


Gambar 1. Anatomi singly linked list.

2.2 Pointer head

Pointer head menyimpan alamat node pertama. List kosong ditandai dengan head yang bernilai nullptr.

```
Node* head = nullptr;
```



Gambar 2. List kosong dan list yang memiliki satu node.

Keadaan	Aturan
List kosong	head == nullptr
List tidak kosong	head menunjuk node pertama
Node terakhir	next == nullptr

2.3 Membuat node

Node dibuat menggunakan new. Nilai disimpan pada data, sedangkan next ditetapkan sesuai hubungan yang akan dibentuk.

```
Node* buatNode(int nilai) {
    return new Node{nilai, nullptr};
}
```

2.4 Membentuk list secara manual

Pembentukan manual memperlihatkan bahwa urutan list ditentukan oleh hubungan next, bukan oleh letak fisik node dalam memori.

```
Node* pertama = buatNode(10);
Node* kedua   = buatNode(20);
Node* ketiga  = buatNode(30);

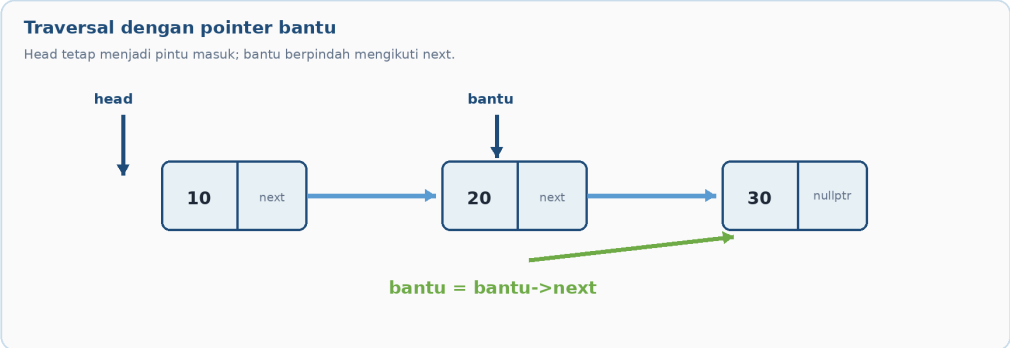
pertama->next = kedua;
kedua->next = ketiga;

Node* head = pertama;
head -> [10] -> [20] -> [30] -> nullptr
```

2.5 Traversal

Traversal mengunjungi node satu per satu mulai dari head. Pointer bantu dipindahkan mengikuti next sampai mencapai nullptr.

```
void tampilkan(const Node* head) {
    const Node* bantu = head;
    while (bantu != nullptr) {
        std::cout << bantu->data << ' ';
        bantu = bantu->next;
    }
    std::cout << '\n';
}
```



Gambar 3. Traversal menggunakan pointer bantu tanpa memindahkan head.

2.6 Aturan dasar list

Hubungan antarnode harus tetap valid sebelum dan sesudah setiap operasi.

1. head bernilai nullptr jika list kosong;
2. head menunjuk node pertama jika list tidak kosong;
3. setiap next menunjuk node berikutnya atau nullptr;
4. node terakhir mempunyai next == nullptr;
5. seluruh node dapat dicapai dari head;
6. tidak terdapat siklus; dan
7. node yang dihapus dibebaskan tepat satu kali.

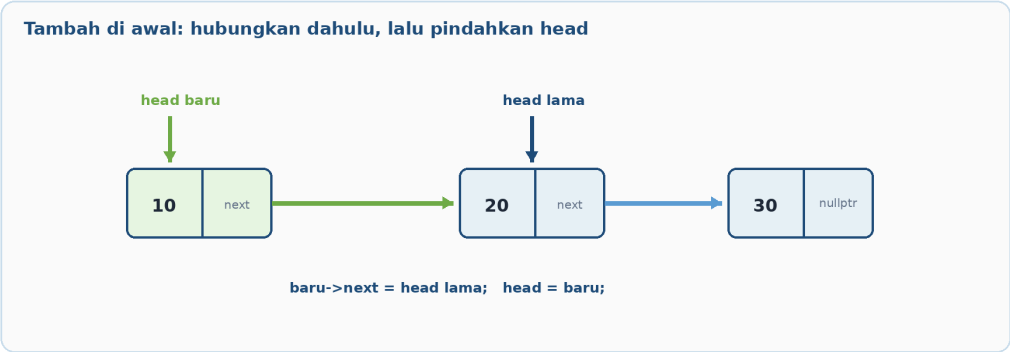
3. Operasi Singly Linked List

Operasi linked list mengubah hubungan pointer. Setelah operasi selesai, periksa kembali posisi head, hubungan next, dan node terakhir.

3.1 Menambah node di awal

Node baru dihubungkan ke node pertama lama, kemudian head dipindahkan ke node baru.

```
void tambahAwal(Node*& head, int nilai) {
    Node* baru = new Node{nilai, head};
    head = baru;
}
```



Gambar 4. Penambahan node di awal list.

Node*& head: Reference terhadap pointer digunakan karena fungsi mengubah head milik pemanggil.

3.2 Menambah node di akhir

Pada list kosong, node baru langsung menjadi head. Pada list tidak kosong, pointer bantu berjalan menuju node terakhir sebelum link baru dipasang.

```
void tambahAkhir(Node*& head, int nilai) {
    Node* baru = new Node{nilai, nullptr};
    if (head == nullptr) {
        head = baru;
        return;
    }

    Node* bantu = head;
    while (bantu->next != nullptr)
        bantu = bantu->next;

    bantu->next = baru;
}
```

3.3 Mencari data

Pencarian memeriksa data setiap node mulai dari head. Fungsi mengembalikan pointer ke node pertama yang cocok atau nullptr jika data tidak ditemukan.

```
Node* cari(Node* head, int nilai) {
    Node* bantu = head;
    while (bantu != nullptr) {
        if (bantu->data == nilai)
            return bantu;
        bantu = bantu->next;
    }
    return nullptr;
}
```

3.4 Menghapus data

Penghapusan menggunakan pointer target dan pointer sebelum. Hubungan list diperbaiki sebelum target dibebaskan.

```
bool hapusData(Node*& head, int nilai) {
    if (head == nullptr) return false;

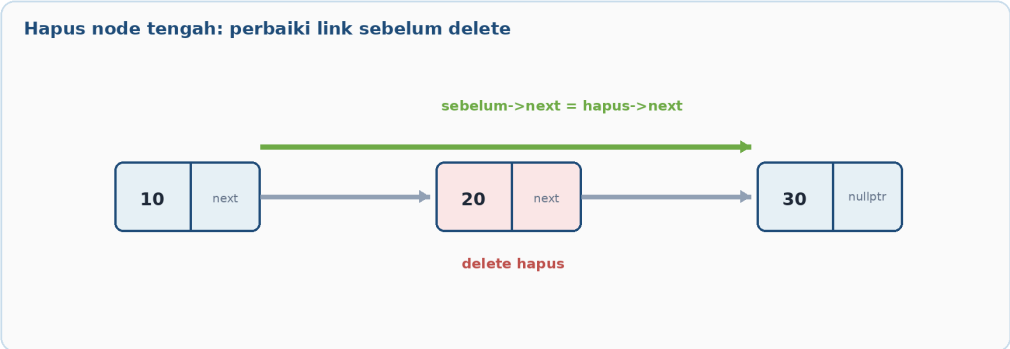
    Node* target = head;
    Node* sebelum = nullptr;

    while (target != nullptr && target->data != nilai) {
        sebelum = target;
        target = target->next;
    }

    if (target == nullptr) return false;

    if (sebelum == nullptr)
        head = target->next;
    else
        sebelum->next = target->next;

    delete target;
    return true;
}
```

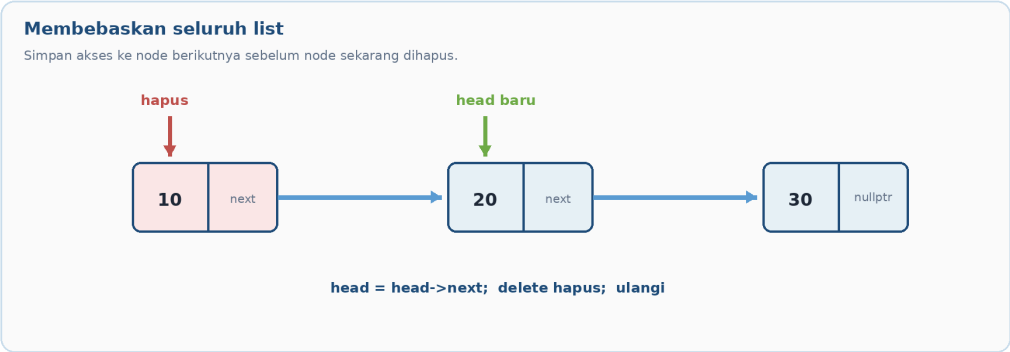


Gambar 5. Link diperbaiki sebelum node target dihapus.

3.5 Membebaskan seluruh list

Clear menghapus node satu per satu dari awal. Head dipindahkan sebelum node lama dibebaskan.

```
void clear(Node*& head) {
    while (head != nullptr) {
        Node* hapus = head;
        head = head->next;
        delete hapus;
    }
}
```



Gambar 6. Pembebasan seluruh node sampai head bernilai nullptr.

4. Kegiatan Praktikum

Kerjakan kegiatan secara berurutan. Gunakan satu berkas latihan dan tampilkan keadaan list setelah setiap operasi.

4.1 Membentuk dan menelusuri list

Gunakan struktur Node pada Subbab 2.1, fungsi buatNode pada Subbab 2.3, dan pembentukan list pada Subbab 2.4. Simpan dalam berkas latihan_sll.cpp.

1. bentuk list 10 -> 20 -> 30;
2. tampilkan alamat setiap node dan nilai next;
3. panggil fungsi tampilkan dari Subbab 2.5;
4. verifikasi bahwa head tetap menunjuk node pertama; dan
5. panggil clear sebelum program berakhir.

4.2 Menambah node

Tambahkan fungsi pada Subbab 3.1 dan 3.2 ke dalam latihan_sll.cpp.

1. uji tambahAwal pada list kosong dan tidak kosong;
2. uji tambahAkhir pada list kosong, satu node, dan beberapa node; serta
3. tampilkan list setelah setiap operasi.

4.3 Mencari dan menghapus node

Tambahkan fungsi pada Subbab 3.3 dan 3.4 ke dalam latihan_sll.cpp.

1. cari data yang tersedia dan tidak tersedia;
2. hapus node pertama, tengah, dan terakhir;
3. hapus satu-satunya node;
4. hapus data yang tidak tersedia; dan
5. tampilkan list setelah setiap operasi.

5. Tugas Praktikum

Buat daftar mahasiswa menggunakan singly linked list. Seluruh bagian program disimpan dalam satu berkas bernama modul3_mahasiswa.cpp.

1. menambah mahasiswa di akhir list;
2. menampilkan seluruh mahasiswa;
3. mencari mahasiswa berdasarkan NIM;
4. menolak NIM duplikat;
5. menghapus mahasiswa berdasarkan NIM;
6. menampilkan jumlah mahasiswa; dan
7. membebaskan seluruh node sebelum program berakhir.

5.1 Kerangka program

Salin kerangka berikut ke modul3_mahasiswa.cpp. Deklarasi fungsi ditempatkan sebelum main, sedangkan definisi fungsi ditambahkan setelah main pada berkas yang sama.

```
#include <iostream>
#include <string>

struct Mahasiswa {
    std::string nim;
    std::string nama;
    double ipk;
};

struct Node {
    Mahasiswa data;
    Node* next;
};

bool nimTersedia(const Node* head, const std::string& nim);
bool tambahAkhir(Node*& head, const Mahasiswa& data);
Node* cariNim(Node* head, const std::string& nim);
bool hapusNim(Node*& head, const std::string& nim);
void tampilkan(const Node* head);
int jumlahData(const Node* head);
void clear(Node*& head);
int main() {
    Node* head = nullptr;
    int pilihan;

    do {
        std::cout << "1. Tambah\n2. Tampilkan\n3. Cari\n";
        std::cout << "4. Hapus\n5. Jumlah\n6. Keluar\n";
        std::cin >> pilihan;

        // panggil fungsi sesuai pilihan

    } while (pilihan != 6);

    clear(head);
    return 0;
}
```

5.2 Fungsi yang dilengkapi

Tambahkan definisi fungsi berikut setelah main. Fungsi tampilkan, jumlahData, dan menu dapat dilengkapi berdasarkan traversal pada Subbab 2.5.

```
bool nimTersedia(const Node* head,
                 const std::string& nim) {
    // lengkapi
    return false;
}

bool tambahAkhir(Node*& head,
                 const Mahasiswa& data) {
    // tolak NIM duplikat sebelum membuat node
    // lengkapi
    return true;
}

Node* cariNim(Node* head, const std::string& nim) {
    // lengkapi
    return nullptr;
}

bool hapusNim(Node*& head, const std::string& nim) {
    // lengkapi
    return false;
}

void clear(Node*& head) {
    // lengkapi
}
```

6. Pengujian

Jalankan program dengan seluruh skenario berikut. Catat hasil aktual dan perbaiki program jika hasilnya tidak sesuai.

No.	Skenario	Hasil yang diharapkan
1	List baru	head == nullptr
2	Tambah pada list kosong	Node menjadi head
3	Tambah beberapa mahasiswa	Urutan sesuai pemasukan
4	NIM ditemukan	Node yang benar
5	NIM tidak ditemukan	nullptr
6	NIM duplikat	Penambahan ditolak
7	Hapus node pertama	head diperbarui
8	Hapus node tengah/terakhir	Link tetap valid
9	Hapus satu-satunya node	head == nullptr
10	Hapus data tidak tersedia	List tetap

No.	Skenario	Hasil yang diharapkan
11	clear dan clear kedua	Aman; head == nullptr

6.1 Diagnosis kode

Setiap potongan kode berikut mengandung kesalahan pointer atau pengelolaan memori. Tentukan kesalahan, akibat, dan perbaikannya.

```
// A: head tidak berubah pada pemanggil
void tambahAwal(Node* head, int nilai) {
    Node* baru = new Node{nilai, head};
    head = baru;
}

// B: rantai lama terputus
Node* baru = new Node{10, nullptr};
head = baru;

// C: use-after-delete
delete target;
head = target->next;

// D: memory leak
head = nullptr;
```